

CS 610 GPU Programming (Winter 2026)

Sample Midterm Questions

Section A – True or False (1 point each)

1. Multiple thread blocks can reside on the same SM simultaneously. **True.**
2. Warp divergence across different thread blocks causes serialization. **False. Divergence only matters within a warp.**
3. Local memory is physically located in DRAM. **True. It is per-thread but stored in global memory (DRAM).**
4. `cudaMemcpyAsync` using pageable memory is always asynchronous relative to the host. **False. Only guaranteed asynchronous with pinned memory.**
5. Shared memory and L1 cache use the same on-chip physical memory space. **True. They share configurable on-chip SRAM.**
6. A warp is the basic scheduling unit of a GPU. **True.**
7. A kernel launch blocks the host thread until execution finishes. **False. Kernel launch is asynchronous.**
8. Increasing register usage per thread may reduce occupancy. **True.**
9. Memory transactions are 128 bytes on modern GPUs. **False. Memory transaction = 32B.**
10. Unified Virtual Addressing (UVA) automatically migrates data between CPU and GPU. **False. Unified Memory performs migration.**

Section B – Multiple Choice (Select ALL correct answers)

1. A kernel launches 40 thread blocks, each with 256 threads. How many warps are there in total?

- (a) 80
- (b) 320
- (c) 2560
- (d) 1280

Warps per block = $256 / 32 = 8$

Total warps = $40 \times 8 = 320$

Answer: (B)

2. Execution serialization can be caused by divergence:

- (a) Within a warp
- (b) Across different warps
- (c) Across different thread blocks
- (d) Across CPU and GPU

Answer: (A)

3. Which memory is typically the fastest?

- (a) L2 cache
- (b) Shared memory
- (c) Registers
- (d) Global memory

Answer: (C)

4. A warp loads 32 floats (4 bytes each) starting from a 32-byte aligned address. How many memory transactions are required?

- (a) 1
- (b) 2
- (c) 4
- (d) 8

$32 \text{ threads} \times 4\text{B} = 128\text{B}$

$128\text{B} / 32\text{B per transaction} = 4$

Answer: (C)

5. A GPU has peak throughput of 32 IPC for FP32 add. Latency per FP32 add = 4 cycles. Required concurrency to reach peak?

- (a) 8
- (b) 32
- (c) 128

(d) 64

Concurrency = Latency $\times$ Throughput = $4 \times 32 = 128$

Answer: (C)

6. Which access pattern gives best memory efficiency?

- (a) stride = 1
- (b) stride = 2
- (c) stride = 32
- (d) random access

Answer: (A)

7. Which may cause a block-level deadlock?

- (a) `if (threadIdx.x < 16) __syncthreads();`
- (b) `if (blockIdx.x < 4) __syncthreads();`
- (c) All threads call `__syncthreads()`
- (d) Some threads return before `__syncthreads()`

Answer: (A) and (D)

8. Which operations are asynchronous relative to host?

- (a) Kernel launch
- (b) `cudaMemcpy`
- (c) `cudaMemcpyAsync` (pinned)
- (d) `cudaMalloc`

Answer: (A) and (C)

Section C – Short Answer

1. Explain SIMD, SIMT, and warp divergence.

SIMD: Single instruction on multiple data elements.

SIMT: Single instruction multiple threads, each with its own program counter (PC).

Warp divergence: Threads in a warp take different control paths, causing serialization.

2. What is automatic scalability in CUDA?

CUDA kernels are independent of SM count.

More SMs → more blocks run simultaneously → higher performance without code changes.

3. Explain latency, throughput, concurrency, and latency hiding in the context of executing GPU instructions.

Latency: time to complete an instruction.

Throughput: instructions completed per cycle.

Concurrency: number of instructions in flight.

Latency hiding: overlapping execution to mask long latency.

4. Why does strided memory access reduce memory efficiency?

Strided access spreads accesses across multiple cache lines, increasing memory transactions and reducing bus utilization.

Section D – Problem Solving (Show all work)

Problem 1 – Memory Transaction Analysis

You are optimizing a scientific simulation kernel that processes a very large 1D array. The kernel is launched as:

```
copy<<<1, 1024>>>(data );
```

Inside the kernel, each thread loads one **double** (8 bytes) from global memory.

Assume:

- Global memory transaction size = 32 bytes
- Address is 32-byte aligned
- Data size is sufficiently large

1. How many global memory read transactions are generated?

$$1024 \times 8\text{B} = 8192\text{B}$$

$$\text{Transactions} = 8192 / 32 = 256$$

2. What is the memory efficiency?

$$\text{Efficiency} = 8192 / (256 \times 32) = 100\%$$

3. If instead each thread loads one **float**, how many transactions occur?

$$\text{If float (4B): } 1024 \times 4 = 4096\text{B}$$

$$\text{Transactions} = 4096 / 32 = 128$$

Problem 2 – 3D Grid Configuration for Medical Imaging

You are processing a 3D MRI volume with dimensions:

$$X = 96, \quad Y = 200, \quad Z = 300$$

Each thread processes one voxel.

Block size = $8 \times 10 \times 5$

1. What grid dimensions should be launched?

$$\text{X: } \text{ceil}(96/8) = 12$$

$$\text{Y: } \text{ceil}(200/10) = 20$$

$$\text{Z: } \text{ceil}(300/5) = 60$$

$$\text{Grid} = (12, 20, 60)$$

2. What is the total number of threads launched?

$$\text{Blocks} = 12 \times 20 \times 60 = 14400$$

$$\text{Threads per block} = 400$$

$$\text{Total threads} = 14400 \times 400 = 5,760,000$$

3. How many threads perform no useful work due to boundary padding?

$$\text{Actual data size} = 96 \times 200 \times 300 = 5,760,000$$

$$\text{Total launched threads} = 5,760,000$$

$$\text{No wasted threads} = 0$$

Problem 3 – Latency Hiding in HPC Kernel

Hardware characteristics:

- FP64 multiply latency = 8 cycles
 - Peak FP64 throughput = 0.5 IPC
1. What instruction-level concurrency is required to reach peak throughput?
 $\text{Concurrency} = \text{Latency} \times \text{Throughput}$
 $= 8 \times 0.5 = 4$
 2. If each warp can issue at most 1 instruction per cycle, what does this imply about the minimum number of active warps required?
Need at least 4 independent FP64 multiplies in flight.
Requires multiple active warps to provide enough independent instructions.
 3. Briefly explain how occupancy affects this scenario.
Higher occupancy provides more resident warps,
improving latency hiding and sustaining throughput.

Problem 4 – Parallel Reduction for Climate Data

1. Write pseudocode for block-level shared memory reduction.

```
shared float partial[BLOCK_SIZE];
partial[tid] = input[idx];
__syncthreads();
for (stride = blockDim.x/2; stride > 0; stride /= 2) {
    if (tid < stride) {
        partial[tid] += partial[tid + stride];
    }
    __syncthreads();
}
```

2. Why is `__syncthreads()` required?
Ensures all threads finish updating shared memory
before the next reduction stage begins.
3. What happens if synchronization is removed?
Race conditions occur → incorrect partial sums.

Problem 5 – Tiled 1D Stencil Optimization

$$C[i] = A[i - 1] + A[i] + A[i + 1]$$

1. How does tiling reduce global memory traffic?
Threads cooperatively load a tile into shared memory.
Each element is loaded once instead of three times.
2. Approximately how many global loads are required after tiling?
Approximately N loads instead of $3N$.
3. Why is synchronization required after loading the tile?
Ensures all shared memory data is fully loaded
before computation begins.

Problem 6 – Resource Usage and Occupancy

1. How many registers does one block use?
 $128 \text{ threads} \times 128 \text{ registers} = 16384 \text{ registers}$
2. What limits the number of blocks per SM?
Total SM registers (65536) divided by registers per block (16384) $\rightarrow$ 4 blocks max.
3. How many threads can reside on one SM?
 $4 \text{ blocks} \times 128 \text{ threads} = 512 \text{ threads}$
4. What is the theoretical occupancy?
 $\text{Occupancy} = 512 / 2048 = 25\%$
5. Explain how high register usage impacts performance.
High register usage reduces active warps,
reducing latency hiding and potentially lowering throughput.

Problem 7 – 2D Image Processing Kernel

You are implementing a 2D image processing kernel. Each thread multiplies one pixel by 2.
The image is stored in row-major format:

`image[row * width + col]`

Assume:

- 2D grid and 2D blocks
- Grid dimensions computed using `ceil()`
- Some threads may fall outside valid bounds

(A) Complete the kernel by filling in blanks X, Y, Z, W, Q.

```
--global-- void imageKernel(float* in ,
                             float* out ,
                             int width ,
                             int height) {

    int row = -----X-----;
    int col = -----Y-----;

    if ( -----Z----- ) {

        int idx = -----W-----;

        out[idx] = -----Q-----;
    }
}
```

X: $\text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$
Y: $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$
Z: $\text{row} < \text{height} \ \&\& \ \text{col} < \text{width}$
W: $\text{row} * \text{width} + \text{col}$

Q: $2 * \text{in}[\text{idx}]$

(B) Why is the boundary check necessary?

Because `ceil()` may launch extra threads beyond valid image bounds.

(C) Does this kernel achieve coalesced memory access? Why?

Yes. Threads in x-dimension access consecutive memory addresses.

Problem 8 – Write a Complete CUDA Program (25 points)

You are given two large vectors A and B of length N. Your task is to write a complete CUDA program that computes:

$$C[i] = A[i] + B[i]$$

Requirements:

- Allocate device memory
- Copy data from host to device
- Launch a kernel using block size = 256
- Compute grid size properly
- Copy result back to host
- Free device memory
- Include boundary checking in the kernel
- No error checking required

Assume:

- N is defined
- Host arrays A, B, C are already allocated and initialized

(A) Write the CUDA kernel.

```
__global__ void vecAdd(float* A,
                      float* B,
                      float* C,
                      int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

(B) Write the host code inside `main()`.


```

int main() {

    float *d_A, *d_B, *d_C;
    size_t size = N * sizeof(float);

    // Allocate device memory
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    // Copy inputs to device
    cudaMemcpy(d_A, A, size,
               cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, size,
               cudaMemcpyHostToDevice);

    // Launch kernel
    int blockSize = 256;
    int gridSize =
        (N + blockSize - 1) / blockSize;

    vecAdd<<<gridSize, blockSize>>>(
        d_A, d_B, d_C, N);

    // Copy result back
    cudaMemcpy(C, d_C, size,
               cudaMemcpyDeviceToHost);

    // Free memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}

```

(C) Explain why boundary checking is required in the kernel.

Grid size uses ceiling division.

Some threads may have $\text{idx} \geq N$.

Boundary check prevents illegal memory access.

(D) Does this program achieve coalesced memory access? Why?

Yes. Consecutive threads access consecutive elements of A, B, and C, resulting in coalesced transactions.